Online Examination Network

Final Project Report with Evidence - SDN + Socket Programming (Jackfruit Mini Project)

Team: add names and IDs | Course: add course and section

1. Introduction and objectives

Many students sitting an online exam at the same moment send their traffic to one server, and that traffic shares the network with everything else. When the network is congested, questions arrive late, submissions are slow, and some students are served worse than others. This project builds the examination application on low-level TCP sockets and runs it on a Software-Defined Network (SDN), where a central controller can monitor, protect and manage the exam traffic.

- Implement a concurrent multi-user exam over TCP: login with Student ID and name, question delivery, answer submission, a server-side timer and automatic result calculation.
- Deploy it in a Mininet topology managed by a Ryu (OpenFlow 1.3) controller.
- Let application events influence the network, and let the network react to congestion and link failure.
- Evaluate response time and fairness under congestion and failure against a plain baseline network.

2. System architecture

The network has four switches (five with the backup path), twelve hosts, and a deliberately slow 10 Mbps link between the core switch s1 and the server-side switch s4.

Component	Role
Hosts h1 to h8	Students (exam clients)
Host h9	Exam server, TCP port 5000
Hosts h10 and h12	UDP background traffic source and sink (iperf), used to create congestion
Host h11	Intruder, not on the roster
Switches s1 to s4, plus s5	Open vSwitch; s5 forms a backup path s1-s5-s4
Ryu controller	Topology discovery, routing, monitoring, congestion response, recovery, blocking

The exam server also sends small UDP event messages (START, LOGIN, UNAUTHORIZED, END) to the controller. A web gateway lets students use a browser: the gateway opens one real TCP connection per student to the exam server.

3. Implementation

3.1 Socket application

Messages are JSON objects with a 4-byte length prefix, because TCP is a byte stream with no message boundaries. Client messages: LOGIN, GET_QUESTION, ANSWER, SUBMIT, PING. Server messages: LOGIN_OK, START, QUESTION, ACK, TIME_SYNC, RESULT, PONG, ERROR. The server runs one thread per client with locks on shared state. It checks a roster of allowed IDs, rejects duplicate logins, lets a dropped student reconnect and resume, keeps the authoritative timer, sends periodic time updates, auto-submits when time expires, scores the answers, and writes the results (ID, name, score) to a file. Client clocks are never trusted.

The student client can answer automatically (for experiments) or interactively from the keyboard. The web exam (10 questions, 15 minutes per student) shows a countdown, lets students move between questions, and shows the score with a per-question review after submission.

3.2 SDN controller

- **Routing:** topology is discovered with LLDP, traffic follows shortest paths, and a loop-free flooding tree lets the backup path exist without broadcast storms.
- **Monitoring:** port and flow statistics every 2 seconds, with exam traffic (TCP port 5000) counted as its own class. Every decision is written to an events log.

- **Congestion response:** when a port carries more than 8 Mbps for two consecutive polls, an OpenFlow meter limits all non-exam traffic to 2 Mbps per switch. The limit is lifted after the load has cleared (with a hold time to avoid flapping).
- **Recovery:** on a link failure the controller flushes the affected flows and traffic is re-routed over the surviving path; when the link returns, traffic goes back to the shortest path.
- **Blocking:** when the exam server reports an unauthorized login, the controller installs a drop rule for that host on every switch.
- **Baseline mode:** the same forwarding with no exam class, no limit, no blocking and no reroute, representing a static network.

4. Socket-SDN integration

Application event	Network reaction
START	Logged; exam mode on
LOGIN	Student ID mapped to the host address
UNAUTHORIZED	Drop rule for that host on all switches (shown in Figure 4: BLOCKED 10.0.0.11 one millisecond after the event)
END	Exam mode off

5. Experimental setup and metrics

All runs used the 10 Mbps bottleneck. Conditions: 5, 10 and 20 students, with no background load or a 12 Mbps UDP flood, plus a link-failure scenario (10 students, link s1-s4 down for 20 seconds in the middle of the exam). Each condition was run twice in both baseline and SDN mode, 28 runs in total. Metrics, measured at the client for every request: mean, 95th-percentile and worst request latency, completion rate, and Jain's fairness index over each student's service rate (1 means perfectly equal service).

6. Results

Latencies are in milliseconds, averaged over two runs.

Scenario	Students	Mode	Mean	95th pct	Worst	Jain	Done
12 Mbps flood	5	Baseline	2,844	4,613	50,310	0.77	100%
12 Mbps flood	5	SDN	29	50	287	0.95	100%
12 Mbps flood	10	Baseline	1,334	4,406	22,320	0.78	100%
12 Mbps flood	10	SDN	28	54	284	0.98	100%
12 Mbps flood	20	Baseline	1,466	5,192	43,140	0.86	92.5%
12 Mbps flood	20	SDN	94	249	1,869	0.80	100%
Link failure	10	Baseline	1,482	22,148	33,445	0.99	100%
Link failure	10	SDN	35	48	274	0.98	100%
No load	5	Base / SDN	25 / 27	37 / 41	73 / 88	0.99 / 0.99	100%
No load	10	Base / SDN	24 / 32	35 / 79	57 / 378	1.00 / 0.96	100%
No load	20	Base / SDN	26 / 33	41 / 172	84 / 703	1.00 / 0.99	100%

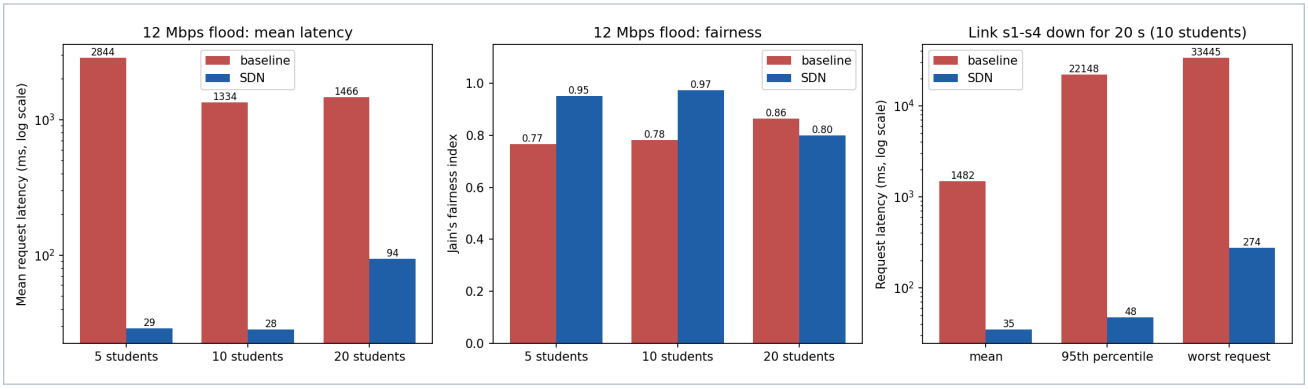


Chart 1. Baseline versus SDN: mean latency and fairness under the 12 Mbps flood, and latency during the link failure (drawn from the averaged results in the table above; values from summary_table.md).

7. Discussion

Under congestion the SDN controller kept the mean request latency near the level of an idle network: 99 times lower than the baseline with 5 students, 47 times with 10 and 16 times with 20. At 20 students the baseline failed to finish 7.5% of the students while SDN completed all of them. On a link failure the controller logged LINK_DOWN and rerouted within milliseconds, so the worst request took 274 ms instead of 33 seconds. The baseline never reroutes, so requests stalled for the whole outage and then for TCP's recovery time. Both modes show a fairness index near 0.98 in the failure test because the outage hurt every student alike, so latency is the meaningful measure there.

The results also show costs and weak points that should be stated openly:

- With no background load, SDN mode is slightly slower than the baseline (about 27 to 33 ms mean against 24 to 26 ms) and has larger worst-case latency at 20 students (703 ms against 84 ms). A likely cause is extra controller handling of the separate exam flows at the start; this was not isolated.
- At 20 students under the flood, SDN fairness (0.80) is lower than the baseline (0.86). A likely cause is the detection delay of about four seconds: a short exam with many students is partly unprotected before the limit applies. The baseline value is also affected because unfinished students drop out of the calculation. Per-student latencies were not examined to confirm this.
- Each condition was repeated only twice, so the numbers are indicative and no statistical test was applied.

8. Limitations and future work

- Fixed congestion threshold and a single shared bottleneck; the meter depends on Open vSwitch support.
- Automated runs answer at random, so their scores are not meaningful. The web gateway checks only the roster ID, with no password.
- Future work: use the server's START event to protect exam traffic before congestion is detected, shorten the detection delay, add per-student rate fairness, and repeat the experiments more times with a statistical comparison.

9. Conclusion

The project delivers a concurrent TCP examination system, including a browser version with a 15-minute timer, running in Mininet under a Ryu controller that monitors traffic, blocks unauthorized hosts, limits background traffic during congestion and recovers from link failure. In 28 controlled runs the SDN approach reduced response time under congestion by a factor of 16 to 99, and under link failure by a factor of more than 40, compared with a static baseline, at a small cost when the network is idle.

10. Evidence (screenshots from the virtual machine)

Each figure below was captured while running the project. Screenshots of the web exam page (login, a question with the countdown, and the result page) were not part of this set; add them here before submission. The LINK_DOWN and REROUTE rows at the very end of Figures 7 and 8 come after APP_END: they are Mininet shutting the network down, not real failures.

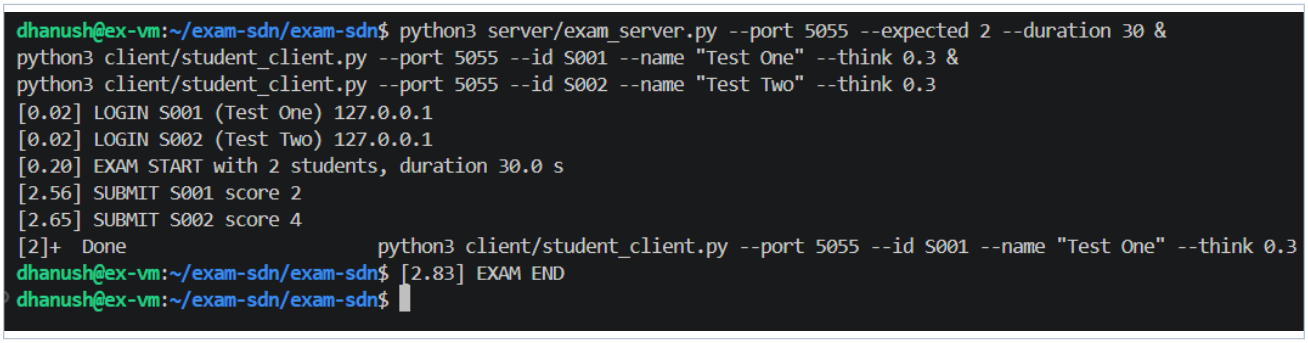


Figure 1. Exam application test: two students log in at the same moment with their names, the exam starts, both submit (scores 2 and 4), and the exam ends. Shows concurrent TCP clients, timer, scoring.

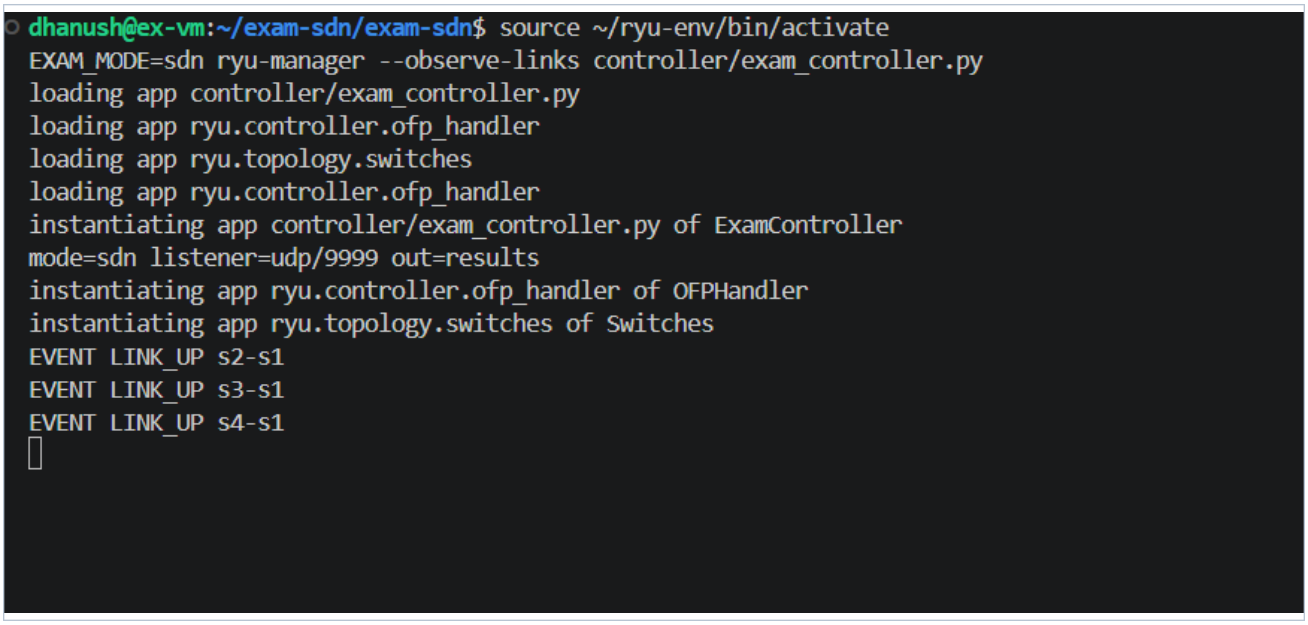


Figure 2. Ryu controller started in SDN mode (listener on UDP 9999) with topology discovery: LINK_UP for s2-s1, s3-s1 and s4-s1.

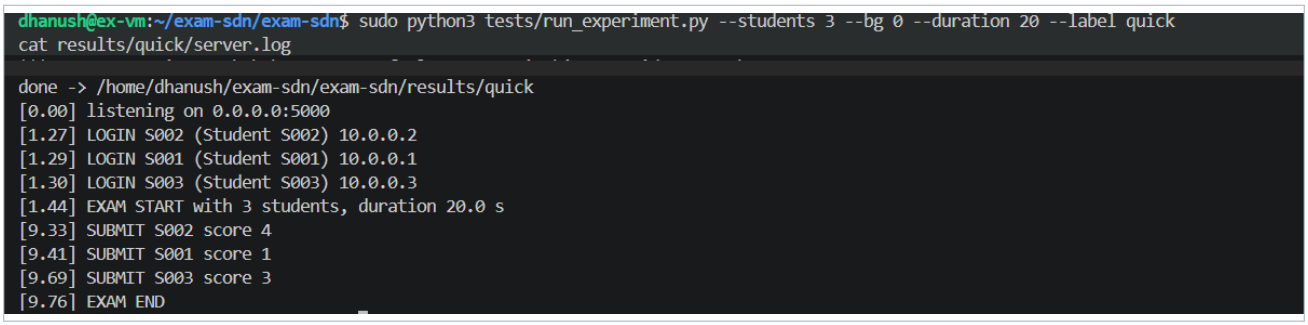


Figure 3. First run inside Mininet: three students log in from hosts 10.0.0.1 to 10.0.0.3 across the SDN-controlled network, the exam starts, all submit and the run completes.

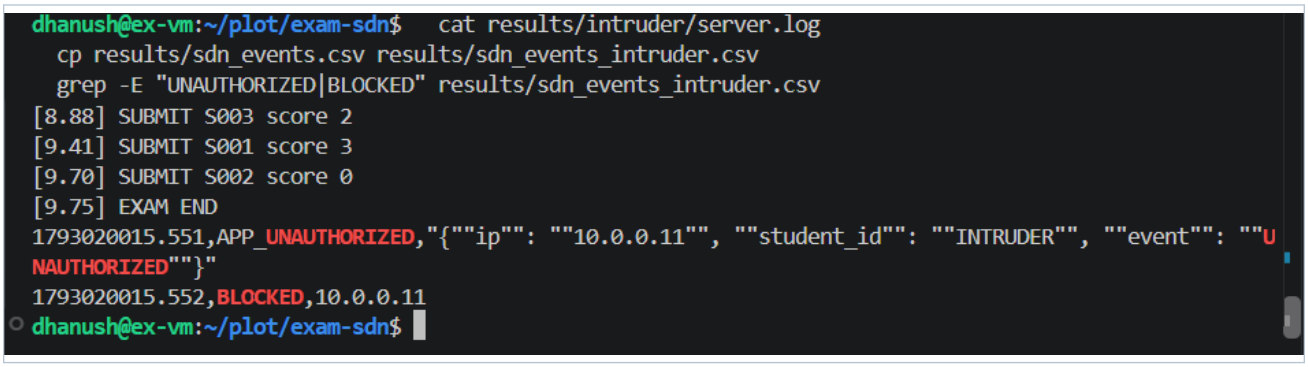


Figure 4. Socket-SDN integration: the exam server reports an unauthorized login from 10.0.0.11 (APP_UNAUTHORIZED) and the controller blocks the host one millisecond later (BLOCKED), while the three real students finish normally.

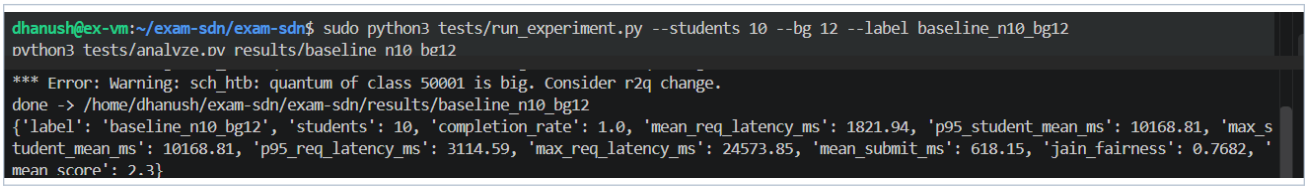


Figure 5. Baseline under a 12 Mbps flood (10 students): mean latency 1,822 ms, 95th percentile 3,115 ms, worst request 24,574 ms, fairness 0.77.

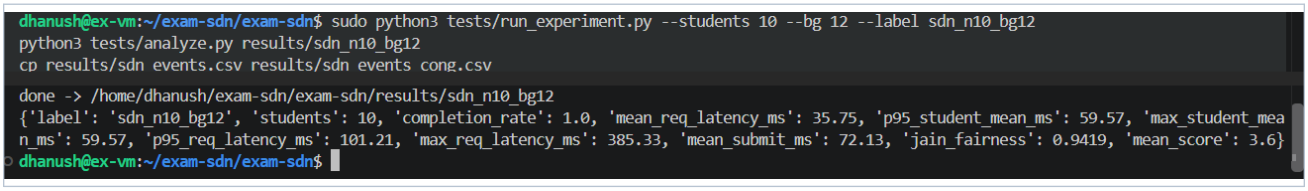


Figure 6. SDN under the same flood: mean latency 36 ms, 95th percentile 101 ms, worst request 385 ms, fairness 0.94.

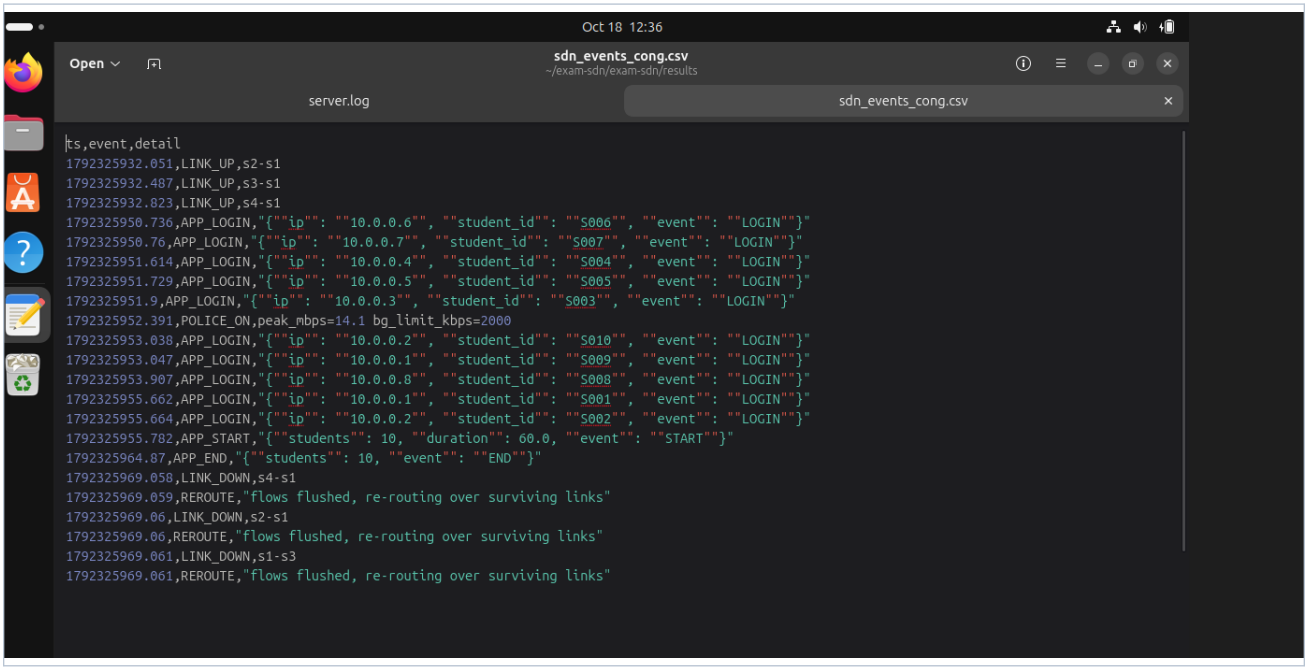


Figure 7. Controller event log of the SDN congestion run: students log in, POLICE_ON is logged at a peak of 14.1 Mbps with a 2,000 kbps background limit, then the exam starts and ends.

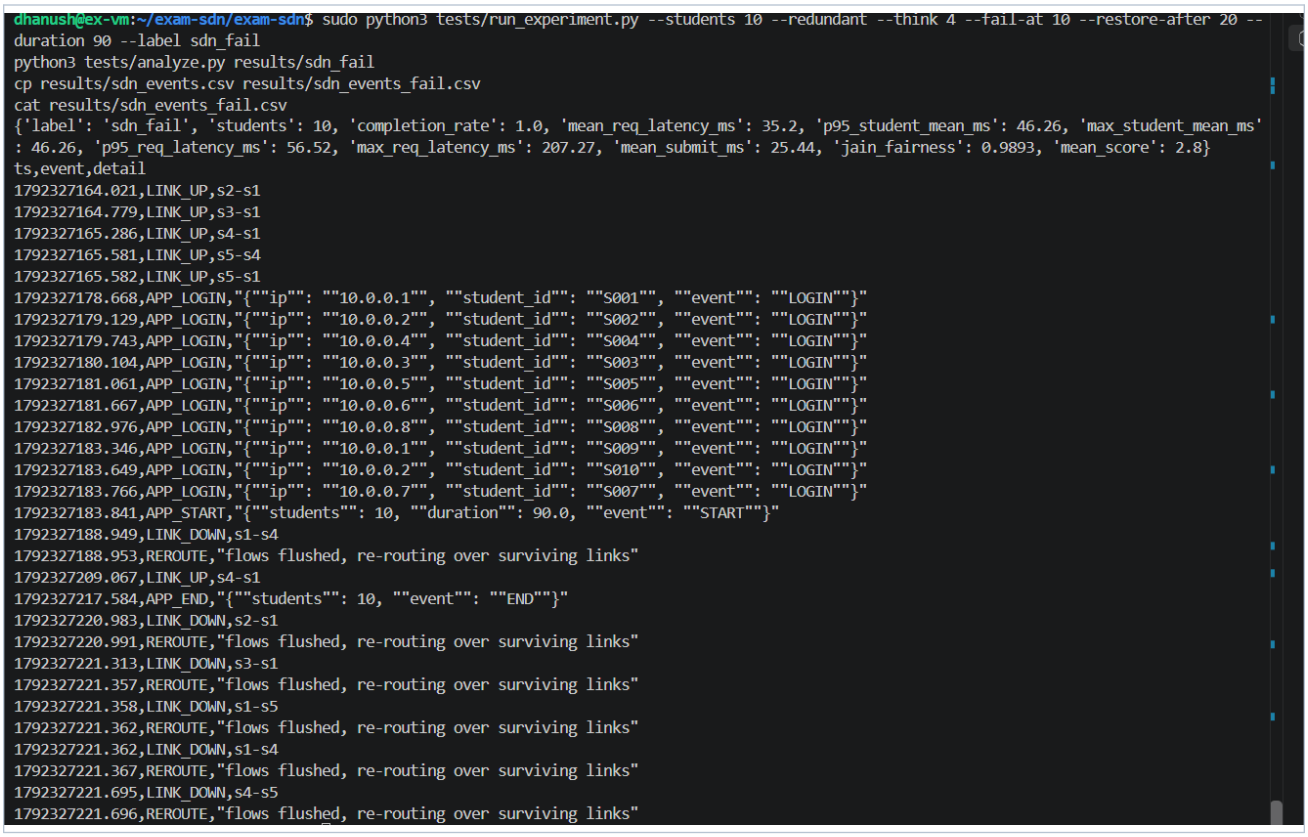


Figure 8. SDN link-failure run: LINK_DOWN s1-s4 is followed by REROUTE 4 ms later, and LINK_UP 20 s later when the link returns. Mean latency 35 ms, worst request 207 ms, all students completed.

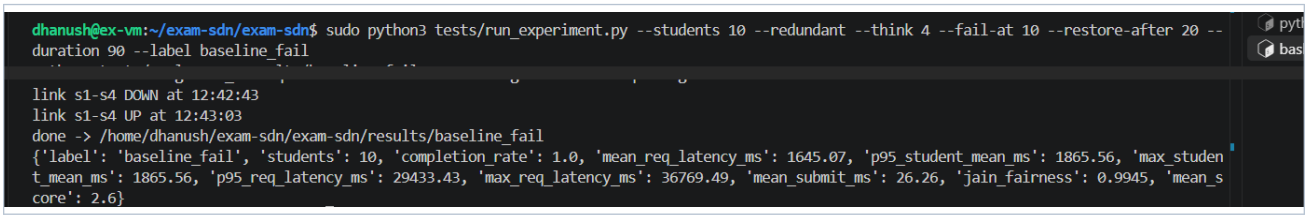


Figure 9. Baseline link-failure run: the link is down for 20 s (12:42:43 to 12:43:03) and nothing reroutes. Mean latency 1,645 ms, 95th percentile 29,433 ms, worst request 36,769 ms.

```
dhanush@ex-vm:~/plot/exam-sdn$ python3 tests/plot.py
cat results/matrix.log
wrote /home/dhanush/plot/exam-sdn/results/plots and summary_table.csv / summary_table.md
| scenario | students | bg_mbps | mode | runs | mean_req_latency_ms | p95_req_latency_ms | max_req_latency_ms |
 jain_fairness | completion_rate |
|---|---|---|---|---|---|---|---|---|---|
| fail | 10 | 0 | baseline | 2 | 1481.83 | 22147.54 | 33444.585 | 0.985 | 1.0 |
| fail | 10 | 0 | sdn | 2 | 34.6 | 47.695 | 273.6 | 0.984 | 1.0 |
| load | 5 | 0 | baseline | 2 | 25.26 | 36.725 | 73.195 | 0.991 | 1.0 |
| load | 5 | 0 | sdn | 2 | 26.615 | 40.93 | 87.87 | 0.994 | 1.0 |
| load | 5 | 12 | baseline | 2 | 2843.54 | 4612.835 | 50310.485 | 0.766 | 1.0 |
| load | 5 | 12 | sdn | 2 | 28.675 | 50.095 | 286.605 | 0.952 | 1.0 |
| load | 10 | 0 | baseline | 2 | 24.105 | 34.865 | 56.665 | 0.997 | 1.0 |
| load | 10 | 0 | sdn | 2 | 32.295 | 78.605 | 377.64 | 0.964 | 1.0 |
| load | 10 | 12 | baseline | 2 | 1334.175 | 4406.1 | 22320.435 | 0.783 | 1.0 |
| load | 10 | 12 | sdn | 2 | 28.1 | 54.055 | 284.375 | 0.975 | 1.0 |
| load | 20 | 0 | baseline | 2 | 25.835 | 40.525 | 84.175 | 0.996 | 1.0 |
| load | 20 | 0 | sdn | 2 | 33.365 | 172.06 | 703.09 | 0.987 | 1.0 |
| load | 20 | 12 | baseline | 2 | 1465.545 | 5192.0 | 43139.97 | 0.864 | 0.925 |
| load | 20 | 12 | sdn | 2 | 93.615 | 249.315 | 1868.87 | 0.801 | 1.0 |

13:23:33 28 runs planned (7 conditions x 2 reps x 2 modes)
13:23:41 controller started in baseline mode
13:23:41 [1/28] baseline_n5_bg0_r1 (attempt 1)
13:24:12 [2/28] baseline_n5_bg0_r2 (attempt 1)
13:24:44 [3/28] baseline_n5_bg12_r1 (attempt 1)
13:26:22 [4/28] baseline_n5_bg12_r2 (attempt 1)
13:27:57 [5/28] baseline_n10_bg0_r1 already done, skipping
13:27:57 [6/28] baseline_n10_bg0_r2 (attempt 1)
13:28:30 [7/28] baseline_n10_bg12_r1 already done, skipping
13:28:30 [8/28] baseline_n10_bg12_r2 (attempt 1)
13:30:06 [9/28] baseline_n20_bg0_r1 (attempt 1)
13:30:47 [10/28] baseline_n20_bg0_r2 (attempt 1)
13:31:22 [11/28] baseline_n20_bg12_r1 (attempt 1)
13:34:36 [12/28] baseline_n20_bg12_r2 (attempt 1)
13:36:18 [13/28] baseline_fail_n10_r1 already done, skipping
13:36:18 [14/28] baseline_fail_n10_r2 (attempt 1)
13:37:58 controller started in sdn mode
13:37:58 [15/28] sdn_n5_bg0_r1 (attempt 1)
```

Figure 10. Evaluation matrix: the run log (28 runs planned, 7 conditions x 2 repetitions x 2 modes) and the table of averaged results produced by plot.py.

11. Reproducing the results

The source code, setup requirements and every command are in the project repository (README) and in the Setup and Execution Guide. The evaluation set is produced by tests/run_matrix.py and the charts by tests/plot.py.